

Bash Scripting — Funzioni, Case, Test Avanzati

Modulo: 1B | **Corso:** Lab Amministrazione di Sistemi T

Prerequisiti: Modulo 1A completato (variabili, if/else, loop, exit code)

VM: `cd ~/sysAdmin-lab && vagrant up --provider=virtualbox && vagrant ssh`

Cos'è una funzione

Una funzione è un blocco di comandi a cui si assegna un nome, in modo da poterlo richiamare più volte senza riscrivere il codice. La struttura è analoga alle funzioni in C: si dichiara una volta, poi si chiama per nome ogni volta che serve.

```
cat > funzione_base.sh << 'EOF'
#!/bin/bash

saluta() {
    echo "Ciao, sono la funzione saluta"
    echo "La data è: $(date)"
}

echo "Prima della chiamata"
saluta
echo "Dopo la chiamata"
saluta
EOF
chmod +x funzione_base.sh
./funzione_base.sh
```

Output atteso:

```
Prima della chiamata
Ciao, sono la funzione saluta
La data è: [data corrente]
Dopo la chiamata
Ciao, sono la funzione saluta
La data è: [data corrente]
```

La funzione viene **dichiarata** (`saluta() { ... }`) e poi **chiamata** scrivendo il suo nome (`saluta`). La definizione deve precedere la chiamata nello script. Le due sintassi equivalenti sono:

```
saluta() { ... }           # forma compatta – quella più usata
function saluta() { ... }  # forma esplicita – identica, solo più verbosa
```

Argomenti di una funzione

All'interno di una funzione, `$1`, `$2`, `#$`, `@$` si riferiscono agli argomenti **della funzione**, non dello script. È una distinzione critica: se lo script viene chiamato con `./script.sh pippo`, `$1` vale `pippo` nel contesto dello script; ma dentro una funzione chiamata come `mia_funzione foo`, `$1` vale `foo` — indipendentemente dagli argomenti dello script.

Per terminare una funzione con un codice di uscita si usa `return N`. La differenza con `exit N` è fondamentale: `exit` chiude l'intero script; `return` chiude solo la funzione e restituisce il controllo al chiamante.

```
cat > funzione_argomenti.sh << 'EOF'
#!/bin/bash

descrivi_file() {
    FILE=$1
    if [ -z "$FILE" ]; then
        echo "Errore: nessun file specificato"
        return 1
    fi
    if [ -f "$FILE" ]; then
        echo "$FILE: è un file regolare"
    elif [ -d "$FILE" ]; then
        echo "$FILE: è una directory"
    else
        echo "$FILE: non esiste"
    fi
}

descrivi_file /etc/hostname
descrivi_file /etc
descrivi_file /file_inesistente
descrivi_file
EOF
chmod +x funzione_argomenti.sh
./funzione_argomenti.sh
```

Output atteso:

```
/etc/hostname: è un file regolare
/etc: è una directory
/file_inesistente: non esiste
Errore: nessun file specificato
```

Variabili locali

Per default, le variabili in Bash sono **globali**: una variabile dichiarata dentro una funzione è visibile anche fuori. Usare `local` limita la visibilità alla funzione. È buona pratica dichiarare sempre `local` le variabili di lavoro interne a una funzione — evita di sovrascrivere per errore variabili dello script.

```
cat > variabili_locali.sh << 'EOF'
#!/bin/bash

funzione_test() {
    GLOBALE="sono globale"
    local LOCALE="sono locale"
    echo "Dentro la funzione: GLOBALE=$GLOBALE, LOCALE=$LOCALE"
}

funzione_test
echo "Fuori dalla funzione: GLOBALE=$GLOBALE"
echo "Fuori dalla funzione: LOCALE=$LOCALE"
EOF
chmod +x variabili_locali.sh
./variabili_locali.sh
```

Output atteso:

```
Dentro la funzione: GLOBALE=sono globale, LOCALE=sono locale
Fuori dalla funzione: GLOBALE=sono globale
Fuori dalla funzione: LOCALE=
```

LOCALE risulta vuota fuori dalla funzione perché non è visibile al di fuori del suo scope.

Restituire una stringa da una funzione

`return` restituisce solo numeri interi (0-255), usati come exit code. Per restituire una stringa da una funzione si usa `echo + command substitution`: la funzione stampa il valore con `echo`, e il chiamante lo cattura con `$(nome_funzione argomento)`.

```
cat > funzione_ritorno.sh << 'EOF'
#!/bin/bash

ottieni_estensione() {
```

```

    local FILE=$1
    echo "${FILE##*.}"
}

ESTENSIONE=$(ottieni_estensione "documento.pdf")
echo "Estensione: $ESTENSIONE"

ESTENSIONE=$(ottieni_estensione "script.sh")
echo "Estensione: $ESTENSIONE"
EOF
chmod +x funzione_ritorno.sh
./funzione_ritorno.sh

```

Output atteso:

```

Estensione: pdf
Estensione: sh

```

Il pattern `${FILE##*.}` è una **parameter expansion**: rimuove dalla variabile tutto ciò che precede l'ultimo `.` compreso, lasciando solo l'estensione. È un pattern utile da riconoscere — una trattazione completa delle parameter expansion è materiale avanzato.

Risposta alla domanda — perché "documento.pdf" è tra virgolette?

Le virgolette doppie proteggono la stringa come argomento singolo. Senza di esse Bash interpreta ogni token separato da spazio come argomento distinto. Per "documento.pdf" non ci sono spazi quindi tecnicamente funzionerebbe anche senza, ma le virgolette sono buona pratica su qualsiasi stringa letterale. Il punto concettuale è diverso: in `ottieni_estensione "documento.pdf"` stiamo passando una stringa letterale come `$1`. In altri punti abbiamo passato `$FILE` — anche quello è un valore che diventa `$1` dentro la funzione, esattamente allo stesso modo. La differenza è solo la sorgente del valore: una volta è una costante letterale, l'altra è una variabile. La funzione non distingue: riceve sempre un `$1`.

Il costrutto case

`case` confronta una variabile contro una serie di pattern, uno alla volta. È più leggibile di una catena `if/elif/elif` quando i confronti riguardano tutti lo stesso valore — analogamente allo `switch` in C.

```

cat > case_base.sh << 'EOF'
#!/bin/bash

```

```

GIORNO=$1

case $GIORNO in
    lunedì|martedì|mercoledì|giovedì|venerdì)
        echo "$GIORNO è un giorno lavorativo"
        ;;
    sabato|domenica)
        echo "$GIORNO è un giorno festivo"
        ;;
    *)
        echo "Giorno non riconosciuto: $GIORNO"
        ;;
esac
EOF
chmod +x case_base.sh
./case_base.sh lunedì
./case_base.sh domenica
./case_base.sh stocazzo

```

Output atteso:

```

lunedì è un giorno lavorativo
domenica è un giorno festivo
Giorno non riconosciuto: stocazzo

```

Struttura generale:

```

case $VARIABILE in
    pattern1)
        # comandi
        ;;
    pattern2|pattern3)
        # comandi – il | è l'OR tra pattern
        ;;
    *)
        # catch-all – eseguito se nessun pattern ha fatto match
        ;;
esac

```

Tre dettagli sintattici obbligatori: - `;;` chiude ogni branch — senza di esso Bash entrerebbe nel branch successivo - `esac` chiude il blocco (case scritto al contrario — stessa convenzione di `fi` per `if`) - `*` come pattern corrisponde a qualsiasi valore — è il default, equivalente al `default:` di C

case su estensioni di file

Un uso classico in SysAdmin è reagire al tipo di file in base all'estensione. Il * nei pattern funziona qui come wildcard di glob: *.sh corrisponde a qualsiasi stringa che termina con .sh.

```
cat > case_file.sh << 'EOF'
#!/bin/bash
FILE=$1

if [ -z "$FILE" ]; then
    echo "Uso: $0 <file>"
    exit 1
fi

case $FILE in
    *.sh)
        echo "$FILE: script Bash"
        ;;
    *.conf|*.cfg)
        echo "$FILE: file di configurazione"
        ;;
    *.log)
        echo "$FILE: file di log"
        ;;
    *.txt|*.md)
        echo "$FILE: file di testo"
        ;;
    *)
        echo "$FILE: tipo non riconosciuto"
        ;;
esac
EOF
chmod +x case_file.sh
./case_file.sh script.sh
./case_file.sh syslog.log
./case_file.sh README.md
./case_file.sh stoca
./case_file.sh
```

Risposta alla domanda — a cosa serve il primo `if [ -z "$FILE" ]`?

È una **guard clause**: verifica che l'utente abbia effettivamente passato un argomento prima di procedere. Se \$FILE è vuota significa che lo script è stato lanciato senza argomenti (es. ./case_file.sh da solo). In quel caso non avrebbe senso entrare nel case — non c'è

nessun file da classificare. Quindi si stampa un messaggio di aiuto che dice come usare correttamente lo script (Uso: ./case_file.sh <file>) e si esce con errore. Il \$0 in quel messaggio è il nome dello script stesso — in questo modo il messaggio è sempre preciso indipendentemente da come si chiama il file.

Test combinati e operatori logici

In Modulo 1A si è usata una condizione per volta. È possibile combinarne più d'una con operatori logici:

```
cat > test_combinati.sh << 'EOF'
#!/bin/bash
FILE=$1

if [ -f "$FILE" ] && [ -r "$FILE" ]; then
    echo "$FILE esiste ed è leggibile"
fi

if [ -z "$FILE" ] || [ ! -e "$FILE" ]; then
    echo "Argomento mancante o file inesistente"
fi
EOF
chmod +x test_combinati.sh
./test_combinati.sh /etc/hostname
./test_combinati.sh /file_inesistente
./test_combinati.sh
```

Operatore	Significato
[cond1] && [cond2]	AND — vero se entrambe le condizioni sono vere
[cond1] [cond2]	OR — vero se almeno una condizione è vera
[! cond]	NOT — inverte la condizione

Nota fondamentale sulla formattazione: && e || stanno **fuori** dalle parentesi quadre, non dentro. [cond1 && cond2] è sintassi sbagliata in Bash standard. La forma corretta è [cond1] && [cond2].

Le doppie parentesi [[]]

Bash offre [[]] come versione estesa e più robusta di []. La differenza principale è che && e || possono essere scritti **dentro** le doppie parentesi, e il pattern matching con wildcard è supportato nativamente.

```
cat > double_bracket.sh << 'EOF'
#!/bin/bash
NOME=$1

if [[ -n "$NOME" && "$NOME" != root ]]; then
    echo "Utente valido e non root: $NOME"
fi

if [[ "$NOME" == *"admin"* ]]; then
    echo "$NOME contiene la parola admin"
fi
EOF
chmod +x double_bracket.sh
./double_bracket.sh vagrant
./double_bracket.sh sysadmin
```

Aspetto	[]	[[]]
AND/OR	[c1] && [c2] (fuori)	[[c1 && c2]] (dentro)
Pattern matching	non supportato	[["\$VAR" == *.sh]] funziona
Variabili non quotate	possono dare errori	gestite correttamente
Portabilità	POSIX — funziona ovunque	solo Bash — non funziona in sh puro

Per script con #!/bin/bash è preferibile usare [[]]. Per script destinati a essere portabili su shell diverse, usare [].

Exit code con \$?

Ogni comando eseguito in Bash restituisce un **exit code** (codice di uscita). \$? contiene l'exit code dell'ultimo comando eseguito. Convenzione universale: 0 = successo; qualsiasi valore diverso da 0 = errore.

\$? deve essere letto **immediatamente** dopo il comando di interesse — il comando successivo lo sovrascrive.

```
cat > exit_code.sh << 'EOF'
#!/bin/bash
```



```
ls /etc/hostname
echo "Exit code di ls: $?"

ls /file_inesistente
echo "Exit code di ls su file inesistente: $?"

grep "root" /etc/passwd
echo "Exit code di grep (trovato): $?"

grep "utente_inesistente" /etc/passwd
echo "Exit code di grep (non trovato): $?"
EOF
chmod +x exit_code.sh
./exit_code.sh
```

Output atteso:

```
/etc/hostname
Exit code di ls: 0
ls: cannot access '/file_inesistente': No such file or directory
Exit code di ls su file inesistente: 2
root:x:0:0:root:/root:/bin/bash
Exit code di grep (trovato): 0
Exit code di grep (non trovato): 1
```

Script completo — diagnostica.sh

```
cat > diagnostica.sh << 'EOF'
#!/bin/bash

# ---- funzioni ----

controlla_servizio() {
    local SERVIZIO=$1
    if systemctl is-active --quiet "$SERVIZIO"; then
        echo " [OK] $SERVIZIO è attivo"
        return 0
    else
        echo " [KO] $SERVIZIO non è attivo"
        return 1
    fi
}
```

```

controlla_file() {
    local FILE=$1
    local TIPO

    case $FILE in
        *.conf) TIPO="configurazione" ;;
        *.log)  TIPO="log" ;;
        *.sh)   TIPO="script" ;;
        *)      TIPO="generico" ;;
    esac

    if [[ -f "$FILE" && -r "$FILE" ]]; then
        echo " [OK] $FILE ($TIPO) – leggibile"
    elif [ -f "$FILE" ]; then
        echo " [!] $FILE ($TIPO) – esiste ma non leggibile"
    else
        echo " [K0] $FILE – non trovato"
    fi
}

# ---- main ----

echo "=== Diagnostica sistema ==="
echo ""

echo ">> Servizi:"
controlla_servizio ssh
controlla_servizio cron

echo ""
echo ">> File critici:"
controlla_file /etc/hostname
controlla_file /etc/ssh/sshd_config
controlla_file /var/log/auth.log
controlla_file /file_inesistente.conf
EOF
chmod +x diagnostica.sh
./diagnostica.sh

```

Risposta alla domanda — come funziona controlla_servizio?

systemctl is-active --quiet "\$SERVIZIO" è un comando di sistema che verifica se il servizio passato come argomento è attualmente in esecuzione. Il flag --quiet sopprime qualsiasi output testuale — il comando non stampa nulla. Quello che fa, invece, è restituire un **exit code**: 0 se il servizio è attivo, 1 (o altro valore non-zero) se non

lo è.

Il punto chiave è che `if` in Bash non richiede obbligatoriamente le parentesi quadre `[ ]`. Le parentesi quadre sono in realtà un comando chiamato `test` — ma qualsiasi comando può essere usato come condizione di un `if`: Bash valuta il suo exit code. `if systemctl is-active --quiet ssh` significa letteralmente “se il comando `systemctl is-active --quiet ssh` restituisce exit code 0, allora...”. È lo stesso meccanismo che si usa con `$?` , solo applicato direttamente come condizione.

Questo pattern — usare un comando direttamente nell’`if` sfruttandone l’exit code — è molto comune negli script SysAdmin: `if ping -c1 host`, `if ssh user@host`, `if grep "pattern" file`, ecc.

Esercizio in autonomia — `classifica_dir.sh`

Tua versione con analisi dei problemi

```
#!/bin/bash

DIR=$1

if [ !-d "$DIR" ]; then          # ← BUG
    echo " [KO] $DIR non è una directory"
    exit 1
fi

classifica_file() {
    local FILE=$1
    local TIPO

    case $FILE in
        *.conf) TIPO="configurazione" ;;
        *.log)  TIPO="log" ;;
        *.sh)   TIPO="script" ;;
        *)      TIPO="generico" ;;
    esac

    echo " $FILE è un file, di tipo $TIPO"
}

for FILE in $DIR/*; do
    if [ -f "$FILE" ]; then
```

```

        classifica_file $FILE
    fi
done

```

Errori presenti:

1. [**!-d "\$DIR"]** — **spazio mancante tra ! e -d**. Bash interpreta **!-d** come un'unica stringa, non come negazione dell'operatore **-d**. L'errore è **[: !-d: unary operator expected**. La forma corretta è **[! -d "\$DIR"]** — il **!** e l'operatore devono essere token separati.
2. **Contatori di file e directory mancanti**. L'esercizio richiedeva di stampare il totale di file e directory trovati. La soluzione è dichiarare due variabili contatore **prima del loop** (**CONTATORE_FILE=0, CONTATORE_DIR=0**), incrementarle dentro il loop con **\$(VAR + 1)**, e stamparle dopo **done**. Avresti dovuto prevederle prima — ma averlo notato autonomamente è già la comprensione corretta.
3. **Funzione dichiarata dopo la guard clause ma prima del main — posizione corretta**. In Bash le funzioni devono essere dichiarate prima di essere chiamate, ma possono stare ovunque prima della prima chiamata. La posizione che hai scelto è accettabile; per leggibilità convenzionale si mettono tutte le funzioni in cima, prima del codice principale.

Versione corretta

```

cat > classifica_dir.sh << 'EOF'
#!/bin/bash

DIR=$1

if [ -z "$DIR" ]; then
    echo "Uso: $0 <directory>"
    exit 1
fi

if [ ! -d "$DIR" ]; then
    echo "[KO] $DIR non è una directory"
    exit 1
fi

# ---- funzioni ----

classifica_file() {
    local FILE=$1
    local TIPO

```

```

    case $FILE in
        *.conf|*.cfg) TIPO="configurazione" ;;
        *.log)        TIPO="log" ;;
        *.sh)         TIPO="script" ;;
        *.txt|*.md)   TIPO="testo" ;;
        *)            TIPO="generico" ;;
    esac

    echo "  $FILE → $TIPO"
}

# ---- main ----

echo "=== Classificazione: $DIR ==="

CONTATORE_FILE=0
CONTATORE_DIR=0

for ELEMENTO in "$DIR"/*; do
    if [ -f "$ELEMENTO" ]; then
        classifica_file "$ELEMENTO"
        CONTATORE_FILE=$((CONTATORE_FILE + 1))
    elif [ -d "$ELEMENTO" ]; then
        CONTATORE_DIR=$((CONTATORE_DIR + 1))
    fi
done

echo ""
echo "File trovati:      $CONTATORE_FILE"
echo "Directory trovate: $CONTATORE_DIR"
echo "Totale elementi:   $((CONTATORE_FILE + CONTATORE_DIR))"
EOF
chmod +x classifica_dir.sh
./classifica_dir.sh ~/Lab1B
./classifica_dir.sh /etc

```

Differenze rispetto alla tua versione: - Guard clause doppia: prima verifica che l'argomento esista (-z), poi che sia una directory (! -d) — due controlli distinti, due messaggi di errore distinti - "\$DIR"/* con \$DIR quotato — protezione da directory con spazi nel nome - Argomento alla funzione quotato: classifica_file "\$ELEMENTO" — stessa ragione - Contatori dichiarati prima del loop e incrementati dentro

Riepilogo sintattico

Sintassi	Funzione
nome() { ... }	Dichiara una funzione
nome arg1 arg2	Chiama una funzione con argomenti
local VAR=valore	Variabile locale alla funzione
return N	Termina la funzione con exit code N
\$(funzione arg)	Cattura l'output di una funzione come stringa
case \$VAR in p) ;; esac	Confronto su pattern multipli
p1\ p2)	OR tra pattern in case
*)	Pattern catch-all in case
[c1] && [c2]	AND tra condizioni (parentesi singole)
[c1] \ \ [c2]	OR tra condizioni (parentesi singole)
[[c1 && c2]]	AND dentro doppie parentesi
[["\$VAR" == *pattern*]]	Pattern matching in doppie parentesi
\$?	Exit code dell'ultimo comando eseguito
if comando	Condizione basata direttamente sull'exit code di un comando

Connessione con Security

Le funzioni sono la struttura portante degli script di enumerazione automatizzata: - Una funzione `scansiona_host()` richiamata in loop su una lista di IP - Un case che reagisce diversamente a seconda della porta trovata aperta (22→SSH, 80→HTTP, 443→HTTPS) - `$?` usato per sapere se nmap ha trovato qualcosa e decidere se procedere con fasi successive - `if systemctl is-active --quiet servizio` — lo stesso pattern di `controlla_servizio` — usato per verificare se un servizio target è attivo prima di tentare un exploit su di esso - `[[ ]]` con pattern matching per filtrare output di tool come nmap o netstat